

DISS Deep Learning

Lab Session 2: Auto-encoders

December 5, 2025

- Start from the code of last session, with the MNIST classifier.
- You shouldn't need ChatGPT for the code, *try to only use it to understand things*. Don't make it write your code! Don't use it in your coding environment.
- Questions about the contents of this lab and the previous one may come up in the exam. (how did you implement it? what loss? what hyperparameters, what architecture? ...)

1 Auto-encoder

We will implement an auto-encoder: for an input image x , the first network is the **encoder** $f_\theta(x) = z \in \mathbb{R}^d$ which casts the image into a latent dimension of size d . The second network is the **decoder** $g_\phi(z) = \hat{x}$ which outputs a reconstruction of the input from the latent vector.

- Implement the encoder.
 - Pick the architecture based on the previous lab: what do you think will work well?
 - Pick the hyperparameters based on the previous lab and your intuition.
- Implement the decoder.
 - Pick the architecture based on the previous lab. If you're using convolutions, look into ConvTranspose2D.
 - Pick the hyperparameters based on the previous lab and your intuition.
- Implement the training loop.
 - what do you think the loss should be?

- pick the optimizer and learning rate based on the previous lab and your intuition.
- think about the batch size and number of epochs to run.
- Run the training
 - visualize the training and testing losses over time
 - visualize the images built by the network over time. You can use matplotlib as in the examples at the start of the notebook.
- Visualize the learned representation
 - if you used $d > 2$, run a 2D Principal Components Analysis on the latent space over the training dataset (or a subset of it).
 - Visualize where different numbers landed in the latent space.
 - Make a linear path in latent space from one number to another, and visualize all the numbers created by the decoder when given these latent vectors as input.
 - Imagine you wanted to generate a random number. Which difficulties would you encounter?

2 Variational Auto-Encoders

2.1 Explanations

Look up Variational Auto-Encoders, e.g. [this post](#). In essence, they try to impose some structure on the latent space so that we know generally where the latent representations should be (around the center, gaussian distribution). If we know where points are likely to be in the latent space, we can sample from them to generate new, random samples of things that should look like digits!

In order to force the latent vectors of each image to overall look like a Gaussian distribution, we use the *reparametrization trick*: instead of using a fixed latent vector z , we sample $z \sim \mathcal{N}(\mu, \sigma)$ where μ and σ are the new outputs of the encoder. Since we are randomly sampling z , this means we will have a different z even if we input the same thing twice!

For now though, this didn't change much - the network can just learn to predict whatever μ it wanted and $\sigma = 0$. The interesting bit follows: we use a **second loss** that constrains $\mathcal{N}(\mu, \sigma)$ to look as much as possible like the centered and normalized Gaussian $\mathcal{N}(0, I)$. This is called a *KL Divergence*, it forces a distribution to look like another. While it sounds scary, in practice it just amounts to forcing $\sigma_i = 1$ and $\mu_i = 0, \forall i \in \{1, d\}$. Intuitively, to minimize μ we want to have a loss term as μ^2 , and to force $\sigma_i = 1$ we would want something like $(\sigma_i - 1)^2$. The math is essentially this idea, but with a slightly more aggressive way of punishing for σ :

$$D_{KL}(\mathcal{N}(\mu, \sigma) \mid \mathcal{N}(0, I)) = \sum_i \mu_i^2 + [\sigma_i^2 - 1 - \log(\sigma_i^2)]$$

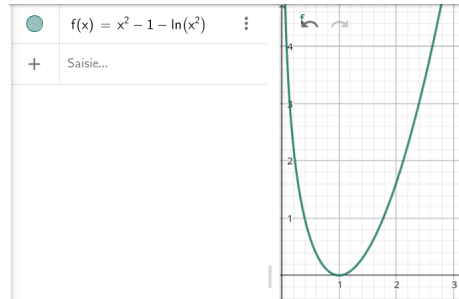


Figure 1: Function to minimize over each σ_i : KL divergence $\sigma_i^2 - 1 - \log(\sigma_i^2)$

I put the term on the right in brackets to show that all three terms concern σ_i . The function to minimize over each σ_i looks like Figure 1.

The -1 term can be removed in the loss - it got there because of the math.

2.2 Doing things

- Create a copy of your notebook and implement the new version of the encoder in the VAE style
- Change your training loop (z calculation, loss) to match the VAE
- Train your networks
- Visualize the latent space learned by the VAE. Do you see a difference with the one from the generic Auto-Encoder?
- Sample some random elements in the latent space using the normal distribution $z \sim \mathcal{N}(0, I)$ and visualize the decoded images.